



An FPGA-Based High-Frequency Trading System for 10 Gigabit Ethernet with a Latency of 433 ns

Created by	 Wonjae Choi
Created time	June 13, 2026 10:18 AM
Category	Paper
Last edited by	 Wonjae Choi
Last updated time	June 14, 2026 11:06 AM

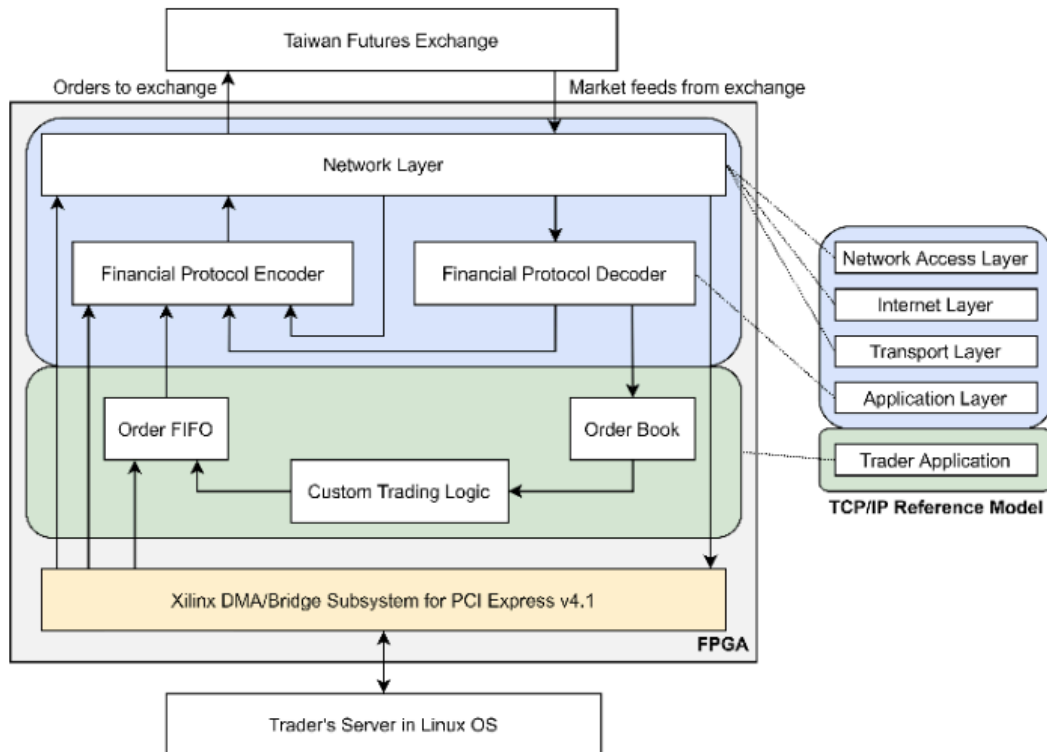
[An_FPGA-Based_High-Frequency_Trading_System_for_10_Gigabit_Ethernet_with_a_Latency_of_433_ns.pdf](#)

Introduction

1. Main Features

- Network stack; 10GbE, 범용 스택 대신 UDP/IP, TCP/IP ... 구현
- Custom financial Protocol decoder; 거래에 영향 주는 메시지만 골라 파싱
- Orderbook - 단일 상품의 top 5 bid/ask + 파생가격만 온칩 메모리에 저장, 상품 추가는 오더북 모듈을 병렬로 더 붙이는 확장 구조
- DMA 서브 시스템 - 수동 주문, 모니터링, 리포팅 과 같은 not Time-sensitive 한 것들은 PCIe 로 호스트 CPU에 workload 배정

Hardware Architecture



1. Financial Protocol Encoder/Decoder

- TCP/UDP Packet 의 header 를 없애고, payload (real message) 파싱/생성하는 것을 의미 (decoding)
- 이 논문은 선택적 파싱을 함으로서, runtime 최적화를 했음 (e.g. 가격변동 메시지, order Id, Quantity)

2. FPGA 에 들어가 있는것

- Network Layer
- Financial Protocol Encode, Decoder
- Orderbook
- Custom Trading Logic

3. FPGA 가 해주는 것: 시장 데이터가 들어오는 순간부터, 주문이 나가는 순간까지의 거래 fast path 를 구현

- CPU, OS, 외부 메모리 거치지 않고 칩 안에서 433ns 를 달성

4. Host CPU 가 하는 것: 느려도 되는 일

- FPGA 가 보내준 데이터를 기반으로, 시장 상황을 보거나 시스템이 정상 동작하는 지 감시

- b. Reporting, Logging
- c. 트레이더가 직접 주문내고 싶을때 host cpu 사용
- d. 거래소 연결에 필요한 설정값 호스트에서 구성해 FPGA 로 전달

Implementation Result

Conclusion

Terms

1. 10GB Ethernet

- a. 초당 10GB 의 데이터를 전송할 수 있는 이더넷 네트워크 규격
- b. Ethernet: 컴퓨터끼리 유선으로 데이터를 주고받는 표준 규칙 (랜선)
 - i. [Destination Addr] [Start MAC] [EtherType] [Payload] [FCS]
 - ii. 컴퓨터 - 랜선- 공유기 - 다른 컴퓨터
 - iii. Application → TCP → IP → Ethernet (내부 용도) → 랜선 (일반)
 - iv. Exchange → Ethernet → NIC → Feed Handler → Strategy (Quant, HFT)
- c. 컴퓨터들이 유선 네트워크로 통신하기 위한 표준 규칙
 - i. 어떤 케이블 사용할지; CAT5e, CAT6, CAT6a
 - ii. 데이터를 어떤 형식으로 보낼지
 - iii. 누가 누구에게 보낼지; MAC Address
 - iv. 어느 속도로 보낼지; 1GbE, 10GbE, 100GbE ...etc

```
#include <array>
#include <stdint>
#include <vector>

struct EthernetFrame {
    std::array<uint8_t, 6> dstMac;    // 목적지 MAC 주소
```

```

std::array<uint8_t, 6> srcMac;    // 출발지 MAC 주소

uint16_t etherType;              // Payload가 뭔지 표시
                                // 0x0800 = IPv4
                                // 0x0806 = ARP
                                // 0x86DD = IPv6

std::vector<uint8_t> payload;    // 실제 데이터

uint32_t fcs;                    // 오류 검사용 CRC
};

```

2. TCP/IP

a. Application → TCP/UDP → IP → Ethernet → 랜선/WiFi

b. Transmission Control Protocol (TCP)

- i. 데이터를 안전하게, 순서대로 전달하자
- ii. 패킷 순서 보장
- iii. 패킷 loss 시 재전송
- iv. 흐름 제어; 처리할 수 있는 속도로 보내게 한다
- v. 연결 관리 (client ↔ Server)

```

class TCPSocket {
public:
    void connect();
    void send(const std::string& data);
    std::string receive();
    void close();
};

```

- c. Ethernet - 같은 네트워크 안에서 누구(MAC)에게 보낼까? ; 아파트 호수 101동 101호
- d. IP - 전 세계에서 어느 컴퓨터 (IP)로 보낼까 ; 서울시 강남구 반포 자이 ..
- e. TCP - 데이터를 안전하고, 순서대로 보낼 수 있을까? ; if "Hello" → TCP Header 추가

f. UDP - 데이터 안전하게 송신하는 것보다, 빠르게 보내는것에 초점

3. Payload

a. 진짜 보내려는 데이터

b. TCP/IP → [TCP/IP Header][Payload]

c. TCP/IP Header

i. 데이터를 전달하기 위해 필요한 부가 정보(metadata)

```
struct IPHeader {  
    uint8_t version;        // IPv4인지 IPv6인지  
    uint8_t ttl;            // 몇 번 라우터를 거칠 수 있는  
지  
    uint16_t totalLength;    // 전체 패킷 길이  
  
    uint32_t srcIP;         // 출발지 IP  
    uint32_t dstIP;         // 목적지 IP  
  
    uint8_t protocol;       // TCP? UDP?  
};
```

```
struct TCPHeader {  
    uint16_t srcPort;        // 출발 포트  
    uint16_t dstPort;        // 목적 포트  
  
    uint32_t seqNum;         // 순서 번호  
    uint32_t ackNum;         // 확인 번호  
  
    bool SYN;  
    bool ACK;  
    bool FIN;  
  
    uint16_t windowSize;  
};
```

4. Media Access Control (MAC)

a. Ethernet 에서 네트워크 장치를 식별하기 위한 48bit (6byte) 주소

b. e.g. 00:1A:2B:3C:4D:5E

5. PCI express (PCIe)

a. CPU ↔ GPU, SSD, NIC, FPGA 를 연결하는 network protocol

6. BRAM (Block RAM)

a. FPGA 내부에 들어 있는 전용 메모리 블록

b. 정보를 저장하는 방식

i. Flip-flop (Register)

ii. BRAM (FPGA 내부 SRAM)

```
reg [31:0] memory [0:1023];  
// Read  
always @(posedge clk)  
    data_out <= memory[addr];  
  
// Write  
always @(posedge clk)  
    if (we)  
        memory[addr] <= data_in;
```

1. 외부 DRAM (DDR)

7. DMA subsystem

a. Direct Memory Access

i. CPU 를 거치지 않고 데이터를 옮긴다는 뜻

b. 원래 데이터를 옮기는 방법 (1000byte 데이터 옮긴다고 가정)

i. 1 byte read → write to memory → read next byte → write to memory
(repeat 1000x)

ii. 이러면 CPU 가 데이터 복사에만 묶인다.

c. DMA 는 이 운반작업을 전담

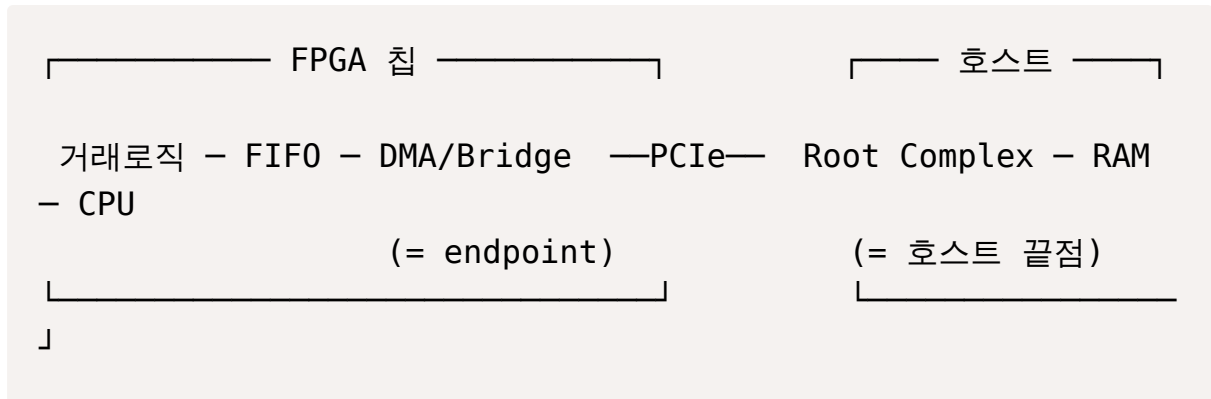
i. 다 하면 CPU 에게 interrupt 로 다 됐다고 notify

ii. 이때 CPU 는 자기 workload 를 한다.

8. 평소 Packet 처리 path

- a. 광신호 → NIC (Network Interface Card) → PCIe → 커널 네트워크 스택 → User Interface → Decoding → Orderbook → strategy decision → kernel → NIC → transmission

9. 이 논문에서 사용하는 packet 처리 path



10. Resource

- a. 외부 연결
 - i. IO: FPGA 외부와 연결되는 pin
 - ii. GT: Gigabit Transceiver - 초고속 직렬 통신 전용 블록
- b. 클럭
 - i. BUFG: Global Clock buffer - 낮은 skw, 지연으로 클럭 분배
 - ii. MMCM: Mixed-Mode Clock Manager; 클럭 생성기, 조정기
- b. 저장
 - i. FF: Flip-flop -1비트를 저장하는 레지스터
 - ii. LUTRAM: LUT 를 메모리 처럼 사용하는 것
 - iii. BRAM
- c. 연산
 - i. LUT: Look Up Table - Combinational Logic 을 구현하는 기본 블록
 - ii. DSP: Multiplication, Addition 전용 하드웨어

11. UDP, TCP 차이

- a. UDP (User Datagram Protocol); 일부 잃어도 빠르게
- b. TCP; 느려도 정확하게